

TRABALLO FIN DE GRAO
GRAO EN ENXEÑARÍA INFORMÁTICA
MENCIÓN EN INGENIERÍA DEL SOFTWARE



Diseño, despliegue y operación de una plataforma cloud para la gestión de transportes en AWS mediante infraestructura como código y prácticas DevOps

Estudiante: Pablo Manzanares López

Dirección: Paula María Castro Castro

A Coruña, mayo de 2026.

Agradecimientos

A mi familia, amistades y profesorado por el apoyo recibido durante el desarrollo de este trabajo. Este apartado se revisará antes de la entrega final para ajustar el tono personal de los agradecimientos.

Resumen

Este Trabajo Fin de Grado aborda el diseño, desarrollo, despliegue y operación de una plataforma cloud para la gestión de transportes sobre AWS. El proyecto se plantea como un backend de alcance funcional acotado, pero con una carga técnica significativa en arquitectura, automatización, infraestructura como código, seguridad básica y validación operativa.

La solución implementa una API REST en ASP.NET Core sobre PostgreSQL para gestionar transportes, vehículos, conductores, usuarios, asignaciones, transiciones de estado y eventos logísticos. La aplicación se empaqueta mediante Docker y se despliega en AWS usando Terraform, ECS Fargate, RDS PostgreSQL, ECR, Application Load Balancer, Route 53, ACM, Secrets Manager, SSM Parameter Store y CloudWatch. También se ha definido un flujo de despliegue manual desde GitHub Actions con autenticación OIDC, evitando claves AWS persistentes.

Como resultado, se ha obtenido un entorno dev real recreable y validado en AWS, con servicio ECS estable, migraciones EF Core ejecutadas como tarea puntual de ECS, verificación de salud, bootstrap de administrador, login con JWT, logs JSON correlables en CloudWatch, dashboard operativo, alarmas Terraform para ALB/ECS/RDS/API y configuración DNS/HTTPS mediante Route 53 y ACM. Tras capturar la evidencia operativa, el entorno dev se destruyó con Terraform para evitar costes residuales, conservando únicamente recursos base de bajo coste como el dominio y la zona DNS.

Abstract

This Bachelor's Thesis addresses the design, development, deployment, and operation of a cloud-based transport management platform on AWS. The project is implemented as a deliberately scoped backend, with a strong technical focus on architecture, automation, infrastructure as code, basic security, and operational validation.

The solution provides a REST API built with ASP.NET Core and PostgreSQL to manage transports, vehicles, drivers, users, assignments, state transitions, and logistics events. The application is packaged with Docker and deployed to AWS using Terraform, ECS Fargate, RDS PostgreSQL, ECR, Application Load Balancer, Route 53, ACM, Secrets Manager, SSM Parameter Store, and CloudWatch. A manual GitHub Actions deployment flow with OIDC authentication has also been defined, avoiding persistent AWS access keys.

The resulting dev environment is reproducible and has been validated on AWS with a stable ECS service, one-off EF Core migrations, health checks, first-admin bootstrap, JWT login, JSON structured logs with request correlation, a CloudWatch dashboard, Terraform-managed alarms for ALB/ECS/RDS/API, and DNS/HTTPS configuration through Route 53

and ACM. After collecting operational evidence, the dev stack was destroyed with Terraform to avoid residual cloud costs, leaving only low-cost foundational resources such as the domain and DNS zone.

Palabras clave:

- Computación en la nube
- DevOps
- Infraestructura como código
- AWS
- Terraform
- ASP.NET Core
- Gestión de transportes

Keywords:

- Cloud computing
- DevOps
- Infrastructure as code
- AWS
- Terraform
- ASP.NET Core
- Transport management

Índice general

1	Introducción	1
1.1	Motivación	1
1.2	Problema a resolver	2
1.3	Alcance del trabajo	2
1.4	Estructura de la memoria	2
2	Contexto y objetivos	3
2.1	Contexto tecnológico	3
2.2	Objetivo general	3
2.3	Objetivos específicos	4
2.4	Criterios de éxito	4
3	Metodología y planificación	6
3.1	Enfoque de trabajo	6
3.2	Fases del proyecto	6
3.3	Gestión del alcance	6
3.4	Ejecución por sprints	7
3.5	Herramientas utilizadas	7
4	Análisis y requisitos	9
4.1	Actores	9
4.2	Requisitos funcionales	9
4.3	Requisitos no funcionales	10
4.4	Modelo de dominio	11
4.5	Casos de uso principales	11
5	Diseño de la arquitectura	13
5.1	Visión general	13

5.2	Arquitectura lógica del backend	13
5.3	Arquitectura de datos	14
5.4	Arquitectura cloud en AWS	14
5.5	Decisiones arquitectónicas	15
6	Implementación del backend	17
6.1	Estructura del proyecto	17
6.2	API REST	17
6.3	Persistencia	18
6.4	Autenticación y autorización	18
6.5	Pruebas del backend	18
7	Infraestructura y despliegue	20
7.1	Contenerización	20
7.2	Infraestructura como código	20
7.3	Entorno AWS	21
7.4	Despliegue de la aplicación	22
7.5	DNS, certificados y HTTPS	22
7.6	Cierre del entorno y control de costes	23
7.7	Gestión de configuración y secretos	24
8	Integración y despliegue continuo	25
8.1	Objetivos del pipeline	25
8.2	Validación automática	25
8.3	Publicación de artefactos	26
8.4	Despliegue automatizado	26
9	Observabilidad y seguridad	28
9.1	Comprobaciones de salud	28
9.2	Logs y métricas	28
9.3	Alarmas y operación	29
9.4	Seguridad aplicada	30
9.5	Limitaciones de seguridad	31
10	Validación y resultados	32
10.1	Estrategia de validación	32
10.2	Pruebas funcionales	32
10.3	Validación de infraestructura	33
10.4	Resultados obtenidos	33

10.5 Evidencia de cierre de coste	34
10.6 Análisis de limitaciones	35
11 Conclusiones	36
11.1 Conclusiones del trabajo	36
11.2 Competencias aplicadas	36
11.3 Lecciones aprendidas	37
11.4 Líneas futuras	37
A Material complementario	40
A.1 Anteproyecto	40
A.2 Evidencias técnicas	40
Lista de acrónimos	43
Glosario	44
Bibliografía	45

Índice de figuras

5.1	Evidencia DNS prevista para el endpoint de desarrollo.	15
7.1	Evidencia prevista de certificado ACM emitido.	23
8.1	Evidencia prevista del pipeline de despliegue completo.	27
9.1	Evidencia prevista de logs JSON correlables.	29
9.2	Evidencia prevista del dashboard operativo.	30

Índice de cuadros

4.1	Requisitos funcionales principales	9
A.1	Capturas recomendadas para la memoria	40

Introducción

Este trabajo presenta el diseño, desarrollo, despliegue y operación de una plataforma cloud para la gestión de transportes. El proyecto se plantea como un trabajo clásico de ingeniería en el que el alcance funcional se mantiene deliberadamente acotado para poder profundizar en decisiones de arquitectura, automatización, [infraestructura como código](#), observabilidad y operación en [Amazon Web Services \(AWS\)](#).

1.1 Motivación

El desarrollo de software actual no se limita a escribir lógica de negocio. Un sistema backend debe poder compilarse, probarse, configurarse, desplegarse, observarse y recuperarse de forma repetible. Esta realidad hace que la ingeniería del software moderna incorpore prácticas de DevOps, infraestructura como código, contenedores, servicios gestionados y automatización de despliegues.

El dominio de gestión de transportes resulta adecuado para este trabajo porque permite modelar entidades y reglas de negocio reales sin convertir el proyecto en un sistema funcionalmente excesivo. Transportes, vehículos, conductores, usuarios, estados y eventos logísticos ofrecen suficiente complejidad para demostrar diseño de API, persistencia, validación, seguridad y trazabilidad, pero mantienen el alcance dentro de un tamaño razonable para un Trabajo Fin de Grado.

La motivación principal del proyecto es construir un sistema pequeño en funcionalidad, pero serio en operación. En lugar de ampliar indefinidamente las características de producto, el trabajo prioriza demostrar que una API backend puede pasar de código fuente a un entorno cloud real en AWS de forma reproducible y documentada.

1.2 Problema a resolver

El problema abordado consiste en proporcionar un backend que permita a un equipo operativo gestionar transportes y sus recursos asociados. El sistema debe permitir crear y consultar transportes, mantener vehículos y conductores, asignar recursos, controlar el ciclo de vida de cada transporte y registrar eventos que aporten trazabilidad.

Además del problema funcional, el trabajo aborda un problema de ingeniería: cómo empaquetar, desplegar y operar esa API en un entorno cloud sin depender de pasos manuales frágiles ni de infraestructura creada de forma informal. Para ello, la infraestructura se define con Terraform, la aplicación se ejecuta como contenedor, la base de datos se despliega como servicio gestionado y los secretos se externalizan.

1.3 Alcance del trabajo

El trabajo se centra en un [backend API](#), persistencia relacional, ejecución local reproducible, infraestructura AWS definida con Terraform, pipeline [CI/CD](#) y mecanismos básicos de observabilidad y seguridad. Quedan fuera del alcance principal el desarrollo de una interfaz de usuario completa y las funcionalidades avanzadas de planificación logística.

1.4 Estructura de la memoria

La memoria se organiza en once capítulos. Tras esta introducción, el capítulo [2](#) presenta el contexto tecnológico y los objetivos del trabajo. El capítulo [3](#) describe la metodología y la planificación seguida. El capítulo [4](#) recoge los requisitos, actores y reglas del dominio. El capítulo [5](#) explica la arquitectura lógica, de datos y cloud. El capítulo [6](#) detalla la implementación del backend. Los capítulos [7](#) y [8](#) tratan la infraestructura, el despliegue y la automatización. El capítulo [9](#) resume las medidas de observabilidad y seguridad. El capítulo [10](#) recoge la validación realizada y los resultados obtenidos. Finalmente, el capítulo [11](#) presenta conclusiones, competencias aplicadas y líneas futuras.

Contexto y objetivos

2.1 Contexto tecnológico

Las aplicaciones backend actuales suelen desplegarse en entornos distribuidos, contenerizados y gestionados por proveedores cloud. Esto permite reducir la administración directa de servidores, pero introduce nuevas responsabilidades: definir redes, permisos, secretos, balanceadores, bases de datos, registros de imágenes, observabilidad y procesos de despliegue.

En este contexto, la **infraestructura como código** permite describir la infraestructura mediante ficheros versionables, revisables y repetibles. Terraform se utiliza en este proyecto para codificar la red, la base de datos, los servicios de ejecución, los repositorios de imágenes, el balanceador de carga, la configuración de DNS y los permisos necesarios [1].

La contenerización proporciona una unidad de despliegue estable entre el entorno local y AWS. La API se empaqueta en una imagen Docker y se ejecuta en ECS Fargate, evitando la gestión directa de servidores. La persistencia se apoya en PostgreSQL gestionado mediante RDS, y el acceso público se concentra en un Application Load Balancer protegido por HTTPS.

El enfoque DevOps aparece en la conexión entre código, validación automática, construcción de imagen, publicación en ECR, despliegue Terraform, migraciones de base de datos y pruebas de humo. El proyecto busca que estas piezas formen un camino verificable desde el repositorio hasta un entorno real.

2.2 Objetivo general

Diseñar, implementar, desplegar y operar una plataforma cloud para la gestión de transportes sobre AWS, utilizando infraestructura como código y prácticas DevOps para garantizar reproducibilidad, automatización, seguridad básica y capacidad de observación.

2.3 Objetivos específicos

- Diseñar una arquitectura cloud para una plataforma de gestión de transportes sobre AWS.
- Implementar un backend con funcionalidades básicas de transportes, vehículos, conductores, usuarios y eventos.
- Definir la infraestructura mediante Terraform para obtener entornos reproducibles.
- Contenerizar la aplicación y desplegarla en servicios gestionados de AWS.
- Automatizar validación, construcción y despliegue mediante CI/CD.
- Incorporar logs, métricas, comprobaciones de salud y trazabilidad operativa.
- Aplicar medidas básicas de seguridad en autenticación, autorización, secretos e infraestructura.
- Evaluar el sistema mediante pruebas funcionales y análisis de ejecución.
- Documentar decisiones, limitaciones y líneas de evolución futura.

2.4 Criterios de éxito

Los criterios de éxito se han formulado como resultados observables:

- La solución compila y las pruebas automatizadas principales se ejecutan correctamente.
- La API permite gestionar el flujo operativo básico: autenticación, usuarios, transportes, vehículos, conductores, asignación, cambios de estado y eventos.
- La base de datos se gestiona mediante migraciones de Entity Framework Core y no mediante cambios manuales no versionados.
- El entorno local puede arrancarse con Docker Compose y configuración externa.
- La infraestructura de AWS se define en Terraform y usa estado remoto S3 con bloqueo DynamoDB.
- La aplicación se despliega como contenedor en ECS Fargate, detrás de ALB y con persistencia en RDS PostgreSQL.
- El entorno dev expone un endpoint HTTPS real en `api.dev.transitops.net`.

- Las migraciones en AWS se ejecutan como tarea ECS puntual, no como efecto lateral opaco del arranque web.
- Los secretos no se almacenan en el repositorio y se consumen desde Secrets Manager.
- Las peticiones son correlables mediante X-Correlation-ID y los logs JSON pueden consultarse en CloudWatch.
- La infraestructura crea dashboard y alarmas operativas mínimas para ALB, ECS, RDS y errores de aplicación.
- El entorno dev puede destruirse con Terraform para eliminar recursos costosos tras capturar evidencias.
- Existe documentación suficiente para explicar arquitectura, despliegue, validación y limitaciones.

Metodología y planificación

3.1 Enfoque de trabajo

El proyecto se desarrolla con un enfoque iterativo e incremental. Cada iteración busca producir una versión funcional o verificable del sistema, evitando introducir complejidad que no aporte valor directo a los objetivos del trabajo.

3.2 Fases del proyecto

1. Análisis y diseño del alcance, requisitos, modelo de datos y arquitectura.
2. Desarrollo del backend y de los casos de uso principales.
3. Contenerización y preparación del entorno local reproducible.
4. Definición de la infraestructura cloud mediante Terraform.
5. Automatización de integración y despliegue continuo.
6. Incorporación de observabilidad y medidas básicas de seguridad.
7. Validación funcional y análisis del comportamiento en ejecución.
8. Redacción de memoria, conclusiones y líneas futuras.

3.3 Gestión del alcance

La gestión del alcance se ha basado en una decisión explícita: construir un producto funcionalmente pequeño, pero técnicamente completo. En vez de añadir módulos de negocio avanzados, como optimización de rutas, facturación, geolocalización en tiempo real o panel

web, se priorizó que el backend mínimo pudiera desplegarse y operarse en AWS de forma creíble.

Esta decisión reduce el riesgo de terminar con una aplicación amplia pero superficial. El valor del trabajo se concentra en demostrar el ciclo completo de ingeniería: modelado de dominio, API, persistencia, pruebas, contenedores, infraestructura cloud, seguridad básica, CI/CD, migraciones y validación operativa.

El alcance funcional se fijó alrededor de transportes, vehículos, conductores, usuarios, asignaciones, estados y eventos logísticos. Cualquier funcionalidad fuera de ese flujo se trató como línea futura salvo que fuese necesaria para demostrar el despliegue o la operación.

3.4 Ejecución por sprints

La planificación del tramo cloud se organizó en sprints semanales. Esta estructura permitió separar claramente la construcción funcional del backend, la definición de infraestructura, el primer despliegue real y el endurecimiento operativo posterior.

Sprint 6 se centró en observabilidad y hardening de despliegue. Su objetivo no fue añadir nuevos casos de uso de negocio, sino convertir el entorno dev en un sistema más diagnosticable y operable. El trabajo incluyó correlación de peticiones, logging JSON, dashboard CloudWatch, alarmas, verificación de observabilidad en el workflow de despliegue, circuit breaker de ECS, parametrización de health checks y ajuste del camino DNS/HTTPS en la cuenta AWS actual. El sprint terminó con una validación real en AWS y con el destroy del entorno para controlar costes.

3.5 Herramientas utilizadas

Las herramientas principales utilizadas han sido:

- ASP.NET Core y .NET 10 para la implementación de la API REST [2].
- Entity Framework Core como ORM y herramienta de migraciones.
- PostgreSQL como base de datos relacional.
- xUnit y WebApplicationFactory para pruebas automatizadas.
- Docker y Docker Compose para empaquetado y ejecución local reproducible.
- Terraform para definir infraestructura AWS como código.
- AWS ECS Fargate, ECR, RDS, ALB, Route 53, ACM, CloudWatch, Secrets Manager y SSM Parameter Store como servicios cloud principales [3, 4, 5, 6].

- GitHub Actions para integración continua y despliegue manual.
- Postman/Newman para pruebas de humo manuales y automatizables.
- LaTeX para la elaboración de la memoria.

Análisis y requisitos

4.1 Actores

El sistema distingue los siguientes actores:

- Usuario no autenticado: puede consultar los endpoints públicos de salud y realizar login, pero no acceder a operaciones de negocio.
- Operador: usuario autenticado encargado de la operación diaria. Puede gestionar transportes, vehículos, conductores, asignaciones, transiciones de estado y eventos logísticos.
- Administrador: usuario autenticado con permisos completos. Además de las operaciones del operador, puede gestionar usuarios, roles y activación de cuentas.
- Plataforma: conjunto de componentes técnicos que interactúan con la API, como Docker, GitHub Actions, ECS, ALB y comprobaciones de salud.

4.2 Requisitos funcionales

Los requisitos funcionales principales son:

Cuadro 4.1: Requisitos funcionales principales

ID	Descripción
RF-01	Exponer endpoints de salud de proceso y de preparación, diferenciando disponibilidad de la aplicación y conectividad con PostgreSQL.
RF-02	Permitir la creación controlada del primer administrador mediante un endpoint de bootstrap protegido por token externo.

RF-03	Autenticar usuarios mediante login con usuario y contraseña, devolviendo un JWT válido.
RF-04	Autorizar endpoints protegidos mediante roles <code>admin</code> y <code>operator</code> .
RF-05	Gestionar usuarios desde endpoints exclusivos para administradores.
RF-06	Gestionar transportes con creación, consulta, actualización, listado filtrado, paginación y borrado lógico.
RF-07	Gestionar vehículos como recursos asignables, incluyendo validación de matrícula y código interno.
RF-08	Gestionar conductores como recursos asignables, incluyendo licencia, contacto y estado activo.
RF-09	Asignar vehículo y conductor a un transporte planificado mediante una acción explícita.
RF-10	Controlar el ciclo de vida de un transporte mediante transiciones de estado válidas.
RF-11	Registrar y consultar eventos logísticos asociados a un transporte, conservando el usuario que los creó.
RF-12	Devolver errores coherentes para validación, autenticación, autorización, recursos inexistentes y conflictos de negocio.

4.3 Requisitos no funcionales

Los requisitos no funcionales se centran en la calidad operativa:

- Reproducibilidad: el entorno local debe poder levantarse con Docker Compose y configuración externa.
- Mantenibilidad: el código debe conservar una estructura simple, con responsabilidades claras y sin dividir el sistema en proyectos innecesarios.
- Persistencia consistente: PostgreSQL y las migraciones de EF Core son la fuente de verdad del esquema.
- Seguridad básica: contraseñas con hash, JWT externo, roles, secretos fuera del repositorio y red privada para ECS y RDS.

- Despliegue reproducible: los recursos cloud deben definirse en Terraform y almacenarse con estado remoto.
- Observabilidad mínima: la API debe exponer salud, emitir logs a CloudWatch y permitir diagnosticar estado del servicio.
- Validación automática: la solución debe compilarse y probarse mediante comandos locales y flujos CI.
- Operación cloud: el entorno dev debe ejecutarse en AWS con HTTPS y base de datos privada.

4.4 Modelo de dominio

El modelo de dominio se articula alrededor de cinco entidades principales:

- **AppUser**: representa usuarios autenticables, con nombre de usuario, correo, hash de contraseña, rol, estado activo y campos de auditoría.
- **Transport**: entidad principal del sistema, con referencia, origen, destino, fechas planificadas y reales, estado y asignación opcional de vehículo y conductor.
- **Vehicle**: recurso material asignable a transportes, identificado por matrícula y, opcionalmente, código interno.
- **Driver**: recurso humano asignable, identificado por licencia y datos básicos de contacto.
- **ShipmentEvent**: evento cronológico vinculado a un transporte y al usuario que lo registra.

Las reglas más relevantes son que un transporte nuevo comienza en estado `planned`, solo puede pasar a `in_transit` si tiene vehículo y conductor asignados, y los estados `delivered` y `cancelled` son terminales. Las entidades operativas aplican borrado lógico, por lo que las restricciones de unicidad se definen sobre registros activos.

4.5 Casos de uso principales

Los casos de uso principales son:

1. Bootstrap del primer administrador: se configura un token externo, se llama al endpoint de bootstrap y se crea el primer usuario `admin` si no existe otro administrador activo.

2. Login: un usuario activo obtiene un **JWT** para acceder a endpoints protegidos.
3. Administración de usuarios: un administrador lista usuarios, crea operadores o administradores, cambia roles y activa o desactiva cuentas.
4. Gestión operativa: un operador crea transportes, vehículos y conductores, consulta listados y actualiza datos básicos.
5. Asignación: se asigna conjuntamente un vehículo y un conductor a un transporte planificado.
6. Ejecución del transporte: se cambia el estado del transporte siguiendo transiciones válidas y se registran eventos logísticos.
7. Consulta de trazabilidad: se recupera el historial cronológico de eventos de un transporte.

Diseño de la arquitectura

5.1 Visión general

La arquitectura global se basa en una API REST monolítica, stateless y contenerizada. El cliente accede al sistema a través de un endpoint público gestionado por un Application Load Balancer, mientras que la ejecución de la aplicación y la base de datos permanecen en subredes privadas. Esta separación permite exponer únicamente el balanceador de carga y mantener PostgreSQL aislado de Internet.

En local, el sistema se ejecuta con Docker Compose, levantando la API y PostgreSQL. En AWS, la API se publica como imagen Docker en ECR y se ejecuta en ECS Fargate. RDS PostgreSQL actúa como almacén persistente. Route 53 resuelve el dominio `api.dev.transitops.net`, ACM emite el certificado TLS y el Application Load Balancer centraliza el tráfico público. La validación cloud admite un fallback HTTP mediante el DNS técnico del ALB cuando se recrea el entorno antes de completar la delegación DNS, pero la configuración objetivo de Terraform activa HTTPS y redirección de HTTP a HTTPS.

La infraestructura se define mediante Terraform, con módulos reutilizables para red, registro de contenedores, base de datos, configuración, observabilidad y runtime de contenedores. Esta decisión permite reproducir el entorno, revisar los cambios de infraestructura como parte del repositorio y destruir el entorno dev tras las validaciones para controlar el coste.

5.2 Arquitectura lógica del backend

El backend se organiza como una solución .NET deliberadamente simple con dos proyectos: `TransitOps.Api` y `TransitOps.Tests`. Dentro de la API se distinguen responsabilidades por carpetas:

- `Controllers`: puntos de entrada HTTP versionados.

- `Contracts`: objetos de petición y respuesta expuestos por la API.
- `Application`: servicios de aplicación que contienen la lógica de casos de uso.
- `Domain`: entidades y enumeraciones del dominio.
- `Infrastructure`: persistencia, configuración EF Core y migraciones.
- `Common, Errors y Middleware`: contrato común de respuestas, errores y tratamiento transversal.

No se ha dividido el backend en múltiples proyectos de dominio, aplicación e infraestructura porque el tamaño del sistema no lo justifica. La separación interna por carpetas ofrece suficiente claridad sin añadir coste accidental.

5.3 Arquitectura de datos

PostgreSQL se utiliza como base de datos principal. El esquema se gestiona mediante migraciones de Entity Framework Core, ubicadas bajo `TransitOps.Api/Infrastructure/Persistence/Migrations`. Esta carpeta es la fuente de verdad del esquema, tanto para local como para cloud.

Las entidades principales aplican campos de auditoría y borrado lógico mediante `deleted_at`. Por ello, varias restricciones de unicidad se han diseñado para registros activos, permitiendo reutilizar identificadores de negocio cuando un registro anterior ha sido eliminado lógicamente. Este criterio aparece en referencias como matrículas de vehículos, licencias de conductores o referencias de transporte.

Los estados y tipos de evento se almacenan como `smallint` con restricciones `check`. Esta solución evita depender de enumerados nativos de PostgreSQL y simplifica la integración con EF Core, manteniendo a la vez validación a nivel de base de datos.

5.4 Arquitectura cloud en AWS

La arquitectura cloud desplegable en AWS incluye Route 53, ACM, Application Load Balancer, [ECS](#) Fargate, [ECR](#), [RDS](#) PostgreSQL, Secrets Manager, Systems Manager Parameter Store y CloudWatch [3, 4, 5, 6].

El entorno dev se despliega en la cuenta AWS 661000947340, región `eu-west-1`. La red está formada por una VPC con subredes públicas para ALB y NAT, subredes privadas de aplicación para ECS y subredes privadas de datos para RDS. Los grupos de seguridad restringen el flujo a: Internet hacia ALB, ALB hacia ECS y ECS hacia RDS.

La API queda preparada para exponerse como `https://api.dev.transitops.net`. En la cuenta objetivo se dispone del dominio `transitops.net` y de una hosted zone pública de Route 53; durante Sprint 6 se corrigió la delegación de nameservers para que la validación DNS de ACM fuese visible públicamente. Terraform crea el certificado, los registros de validación, el alias `api.dev.transitops.net`, el listener HTTPS y la redirección HTTP a HTTPS cuando `enable_https = true`. La base de datos RDS no es pública y solo acepta tráfico PostgreSQL desde el grupo de seguridad de ECS.

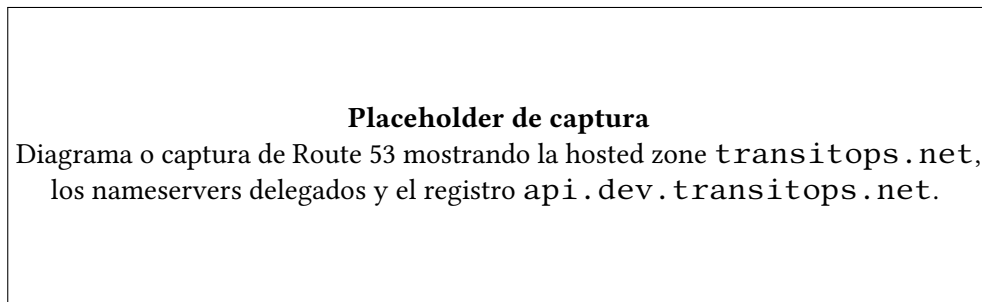


Figura 5.1: Evidencia DNS prevista para el endpoint de desarrollo.

5.5 Decisiones arquitectónicas

Las decisiones arquitectónicas principales han sido:

- Monolito modular frente a microservicios: el dominio y el tamaño del trabajo no justifican la complejidad de microservicios, mensajería o orquestación avanzada.
- ECS Fargate frente a servidores EC2: Fargate reduce administración de infraestructura y encaja bien con una API stateless contenerizada.
- RDS PostgreSQL frente a PostgreSQL autogestionado: RDS simplifica backups, operación y red privada.
- Terraform frente a creación manual: la infraestructura queda versionada, reproducible y revisable [1].
- Secrets Manager y SSM frente a variables hardcodeadas: los secretos no se almacenan en el repositorio y pueden gestionarse de forma separada.
- Migraciones como tarea ECS puntual frente a migraciones automáticas en producción: el despliegue gana control explícito y evita que cada arranque web modifique el esquema.

- OIDC en GitHub Actions frente a claves AWS persistentes: se reduce el riesgo de credenciales de larga duración.
- Observabilidad mínima con CloudWatch frente a plataformas externas: el sprint prioriza logs JSON, correlación, dashboard y alarmas con servicios ya disponibles en AWS, evitando introducir herramientas adicionales antes de que exista una necesidad real.

Estas decisiones favorecen una arquitectura pequeña pero defendible, orientada a demostrar operación real en cloud.

Implementación del backend

6.1 Estructura del proyecto

La solución contiene dos proyectos principales. `TransitOps.Api` implementa el servicio HTTP, la lógica de aplicación, el dominio y la persistencia. `TransitOps.Tests` agrupa las pruebas automatizadas. Esta estructura mantiene bajo el número de proyectos y facilita navegar el código durante el desarrollo y la defensa.

ASP.NET Core se utiliza como framework HTTP [2]. La API define controladores versionados bajo `/api/v1`, un contrato común de respuesta y middleware de errores para homogeneizar las salidas. La configuración se obtiene desde `appsettings`, variables de entorno, secretos locales en desarrollo y servicios AWS en despliegue.

6.2 API REST

La API REST cubre las siguientes áreas:

- Salud: `GET /api/v1/health/live` y `GET /api/v1/health/ready`.
- Autenticación: `POST /api/v1/auth/bootstrap-admin` y `POST /api/v1/auth/login`.
- Usuarios: listado, detalle, creación, cambio de rol y activación/desactivación.
- Transportes: creación, listado filtrado y paginado, detalle, actualización, borrado lógico, asignación y cambio de estado.
- Vehículos y conductores: CRUD con borrado lógico.
- Eventos logísticos: creación y consulta cronológica por transporte.

Las respuestas de éxito se encapsulan en un contrato común. Los errores distinguen validación, autenticación, autorización, no encontrado y conflicto de negocio, usando códigos HTTP coherentes como 400, 401, 403, 404 y 409.

6.3 Persistencia

La persistencia se implementa con Entity Framework Core sobre PostgreSQL. El `TransitOpsDbContext` representa el punto central de acceso a datos y define entidades, relaciones, restricciones e índices. Las migraciones permiten evolucionar el esquema de forma controlada.

El proyecto evolucionó desde una primera aproximación basada en SQL de arranque hacia un modelo donde EF Core migrations es la fuente canónica. Esto simplifica el despliegue y evita divergencias entre entorno local, pruebas y AWS.

En local Docker, la API puede aplicar migraciones al arrancar cuando la opción `Database:ApplyMigrationsOnStartup` está habilitada. En AWS, en cambio, las migraciones se ejecutan mediante el modo `--migrate-only`, como tarea ECS puntual.

6.4 Autenticación y autorización

El sistema incorpora un flujo controlado para crear el primer administrador. El endpoint `POST /api/v1/auth/bootstrap-admin` requiere un token externo configurado fuera del repositorio y solo permite crear el primer usuario administrador si no existe ya un administrador activo.

El login se realiza mediante `POST /api/v1/auth/login`. Las contraseñas se almacenan como hash y, si las credenciales son válidas, la API devuelve un [JWT](#). Este token incluye la identidad y el rol del usuario, permitiendo proteger los endpoints de negocio.

El modelo de autorización distingue `admin` y `operator`. El administrador puede gestionar usuarios; el operador se limita a las operaciones de transporte. Además, la API impide dejar el sistema sin ningún administrador activo.

6.5 Pruebas del backend

Las pruebas del backend se encuentran en `TransitOps.Tests`. Cubren endpoints de salud, transportes, vehículos, conductores, autenticación, usuarios, reglas de estado y flujos de negocio principales. Se utiliza `WebApplicationFactory` para ejecutar pruebas de integración sobre la API.

En el estado actual, la suite principal ejecuta 84 pruebas automatizadas. Estas pruebas se complementan con colecciones Postman/Newman para verificar flujos contra una API viva y con pruebas de humo sobre el entorno AWS desplegado.

Infraestructura y despliegue

7.1 Contenerización

La API se empaqueta en una imagen Docker construida a partir de su `Dockerfile`. El contenedor expone el puerto 8080, instala las dependencias necesarias para el runtime .NET y se configura mediante variables de entorno. La imagen se utiliza tanto para despliegue cloud como para validar el comportamiento de empaquetado.

El entorno local se reproduce con Docker Compose, levantando la API y PostgreSQL. La configuración sensible se proporciona mediante un fichero `.env` no versionado. Esto permite ejecutar el sistema local sin instalar PostgreSQL de forma manual y mantiene una ruta de validación cercana al despliegue real.

7.2 Infraestructura como código

Terraform se organiza bajo `infra/terraform`. La estructura separa un bootstrap de estado remoto, módulos reutilizables y raíces de entorno:

- `bootstrap/remote_state`: crea el bucket S3 de estado y la tabla DynamoDB de bloqueo.
- `modules/platform_foundation`: VPC, subredes, rutas, NAT y grupos de seguridad.
- `modules/container_registry`: repositorio ECR.
- `modules/database`: RDS PostgreSQL.
- `modules/runtime_config`: Secrets Manager y SSM.

- `modules/container_runtime`: ECS, task definition, ALB, listeners, Route 53 y ACM.
- `modules/observability`: grupo de logs, dashboard CloudWatch, alarmas y notificación SNS opcional.
- `modules/github_oidc`: rol de despliegue para GitHub Actions.
- `environments/dev`: composición real del entorno de desarrollo.

El estado remoto se almacena en S3 y se bloquea con DynamoDB. Esto evita estados locales divergentes y reduce el riesgo de cambios concurrentes.

7.3 Entorno AWS

El entorno desplegado utiliza:

- Cuenta AWS: 661000947340.
- Región: `eu-west-1`.
- Entorno: `dev`.
- Dominio previsto: `api.dev.transitops.net`.
- Hosted zone: `transitops.net` en Route 53.
- Identificador de hosted zone: `Z0844787W37HXN9FIJR`.
- Fallback operativo: DNS público del ALB cuando el entorno se recrea antes de completar DNS/ACM.

Los recursos siguen la convención `transitops-dev-<componente>` y se etiquetan con proyecto, entorno, propietario, repositorio, servicio, origen Terraform y las etiquetas `TerraformStack` y `ResourceGroup`. Esta nomenclatura permite identificar recursos y costes, y comprobar tras un `terraform destroy` que no quedan recursos del entorno `dev`.

Como el entorno `dev` es educativo y desechable, la configuración actual desactiva la retención automática de backups de RDS y omite snapshots finales. Esta decisión reduce costes residuales en ciclos frecuentes de creación y destrucción, sin impedir que un entorno futuro de producción defina una política de backup más conservadora.

7.4 Despliegue de la aplicación

El flujo de despliegue aplicado en dev durante la primera validación cloud fue:

1. Crear la infraestructura base con ECS en `desired_count = 0`, evitando arrancar tareas antes de tener imagen en ECR.
2. Construir la imagen Docker de la API.
3. Publicar la imagen en ECR con el tag `sprint4-local-d00aa6b`.
4. Actualizar la task definition para usar la imagen publicada.
5. Cargar los secretos de runtime en Secrets Manager.
6. Ejecutar migraciones con una tarea ECS puntual usando `--migrate-only`.
7. Escalar el servicio ECS a `desired_count = 1`.
8. Activar Route 53, ACM, HTTPS y redirección HTTP a HTTPS.
9. Validar el endpoint público de readiness sobre HTTPS.

En Sprint 6 se repitió la recreación del entorno en la cuenta AWS actual 661000947340. La imagen validada fue `sprint6-local-6bb910c`; se restauraron e importaron secretos que habían quedado programados para eliminación tras un ciclo anterior, se ejecutó una tarea ECS de migración con código de salida 0, se escaló el servicio a `desired = 1` y se validaron readiness, bootstrap de administrador y login. La validación funcional principal de Sprint 6 se hizo mediante el DNS HTTP del ALB, y posteriormente se corrigió la delegación DNS de `transitops.net` en la misma cuenta para que el siguiente `terraform apply` pueda converger directamente con `enable_https = true`.

El módulo de runtime incluye además el deployment circuit breaker de ECS con roll-back, periodo de gracia para health checks y parámetros explícitos para la comprobación del target group: ruta `/api/v1/health/ready`, intervalo, timeout y umbrales de healthy/unhealthy. Esto reduce el riesgo de dejar una versión fallida recibiendo tráfico.

7.5 DNS, certificados y HTTPS

El endpoint objetivo es <https://api.dev.transitops.net>. Para lograrlo, Terraform solicita un certificado público en ACM para `api.dev.transitops.net`, crea el CNAME de validación en Route 53, espera a que el certificado pase a estado ISSUED,

crea el listener HTTPS 443 en el ALB y convierte el listener HTTP 80 en una redirección permanente hacia HTTPS.

Durante la activación en la cuenta 661000947340 apareció una incidencia real de DNS: el dominio registrado `transitops.net` apuntaba a nameservers distintos de los de la hosted zone usada por Terraform. ACM permanecía en `PENDING_VALIDATION` aunque el CNAME existía en Route 53, porque el DNS público no consultaba esa zona. La corrección consistió en actualizar los nameservers del dominio registrado para que coincidieran con la hosted zone `Z0844787W37HXN9FIJR`. Tras la propagación, el CNAME de validación fue visible desde DNS público y el certificado pasó a `ISSUED`.

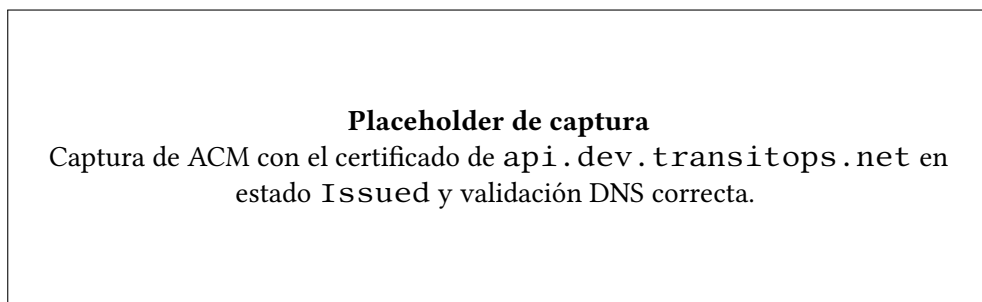


Figura 7.1: Evidencia prevista de certificado ACM emitido.

7.6 Cierre del entorno y control de costes

Después de capturar la evidencia de Sprint 6, el entorno `dev` se destruyó con `terraform destroy` para evitar gasto evitable. La destrucción eliminó ALB, listeners, certificado ACM, registros DNS de `api.dev.transitops.net`, ECS, ECR, RDS, NAT Gateway, VPC, subredes, grupos de seguridad, CloudWatch logs, dashboard, alarmas y secretos del entorno. El repositorio ECR necesitó una limpieza previa de imágenes, ya que AWS no permite eliminar un repositorio no vacío.

Tras el `destroy` se verificó que el state de Terraform quedaba vacío y que no existían recursos costosos asociados al stack `transitops-dev`: ALB, RDS, ECS, ECR, certificado ACM, registros DNS del subdominio, log groups, dashboards, alarmas ni secretos. Permanecen fuera del `destroy` el dominio registrado `transitops.net`, la hosted zone pública y los recursos de backend remoto de Terraform, que son recursos base de bajo coste y necesarios para recrear el entorno.

7.7 Gestión de configuración y secretos

La configuración sensible se externaliza. La cadena de conexión a PostgreSQL, la clave de firma JWT y el token de bootstrap se almacenan en Secrets Manager. Otros parámetros, como emisor, audiencia y expiración del JWT, se almacenan en SSM Parameter Store o se proporcionan como variables no secretas.

En el repositorio solo se incluyen ejemplos y nombres de variables. Los ficheros locales que pueden contener secretos, como `terraform.tfvars`, `backend.hcl` o `.env`, están excluidos del control de versiones.

Integración y despliegue continuo

8.1 Objetivos del pipeline

El objetivo del pipeline es convertir el despliegue en un proceso repetible y auditable. Antes de desplegar, el sistema debe restaurar dependencias, compilar, ejecutar pruebas, construir la imagen Docker y validar Terraform. En el despliegue cloud, además, debe publicar la imagen en ECR, aplicar infraestructura, ejecutar migraciones y realizar pruebas de humo.

El pipeline se diseña para ejecución manual en dev. Esta decisión evita despliegues accidentales durante el desarrollo del TFG y permite capturar evidencias controladas.

8.2 Validación automática

La validación automática incluye:

- `dotnet restore` para restaurar dependencias.
- `dotnet build` para compilar la solución.
- `dotnet test` para ejecutar pruebas automatizadas.
- Validación de migraciones EF Core en el flujo de CI base.
- `docker compose config` para comprobar que la composición local es válida.
- `terraform fmt` y `terraform validate` para asegurar formato y coherencia de infraestructura.
- Validación de recursos de observabilidad tras el despliegue: log group, dashboard CloudWatch, alarmas y presencia de logs correlados.

8.3 Publicación de artefactos

La imagen de la API se construye a partir de `TransitOps.Api/Dockerfile`. En el flujo de despliegue, la imagen se etiqueta con el identificador del commit o con un tag operativo, y se publica en el repositorio ECR `transitops/api`. En el despliegue real de Sprint 4 se publicó la imagen `sprint4-local-d00aa6b`. En la validación de Sprint 6 se publicó y desplegó la imagen `sprint6-local-6bb910c`.

ECR actúa como frontera entre construcción y ejecución: ECS no arranca una versión nueva de la API hasta que la task definition apunta a una imagen disponible.

8.4 Despliegue automatizado

Se han definido dos workflows manuales:

- `terraform-dev.yml`: inicializa Terraform, valida formato y configuración, genera plan y permite aplicar manualmente.
- `deploy-dev.yml`: ejecuta build, tests, construcción de imagen, publicación en ECR, Terraform, migraciones ECS, escalado del servicio, health check, bootstrap de administrador, login y verificaciones mínimas de observabilidad.

GitHub Actions accede a AWS mediante OIDC y asume un rol IAM de despliegue específico para TransitOps:

```
transitops-dev-github-actions-deploy-role
```

No se utilizan claves AWS persistentes en GitHub. La reducción fina de permisos queda como mejora futura, pero el rol ya está limitado al repositorio y rama previstos.

En Sprint 6 se amplió el workflow de despliegue para consumir variables de entorno relacionadas con observabilidad y DNS. La variable `ALARM_EMAIL`, si existe en GitHub, se pasa a Terraform como `TF_VAR_alarm_email` para crear una suscripción SNS de email. Cuando no se configura, Terraform crea las alarmas sin acciones de notificación, evitando depender de una dirección personal para que el entorno sea reproducible.

Después de aplicar infraestructura, el workflow captura outputs relevantes del módulo de observabilidad y comprueba que existen el grupo de logs, el dashboard y las alarmas esperadas. También busca logs JSON con un identificador de correlación generado durante el smoke test. Estas comprobaciones convierten la observabilidad en un criterio de despliegue, no en una revisión manual posterior.

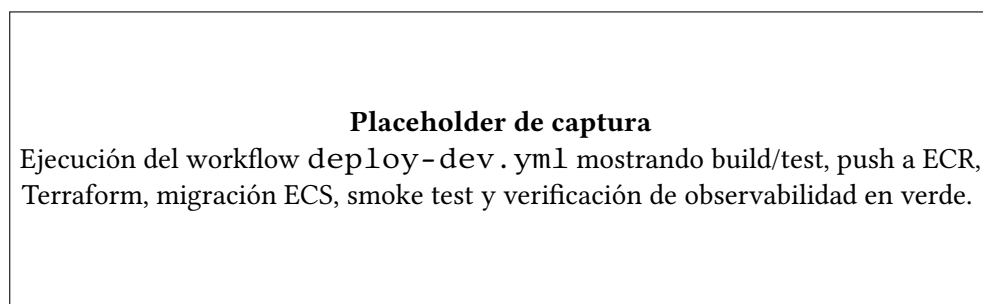


Figura 8.1: Evidencia prevista del pipeline de despliegue completo.

Observabilidad y seguridad

9.1 Comprobaciones de salud

La API expone dos endpoints de salud. `/api/v1/health/live` indica que el proceso está vivo. `/api/v1/health/ready` comprueba la conectividad real con PostgreSQL mediante el `DbContext`. Esta distinción permite diferenciar entre una aplicación arrancada y una aplicación preparada para servir tráfico.

En AWS, el target group del Application Load Balancer utiliza la ruta de readiness para decidir si la tarea ECS puede recibir tráfico. También se ha usado el mismo endpoint como prueba de humo final. En Sprint 6 se fijó esta ruta como valor por defecto parametrizable del target group y se añadió un periodo de gracia de ECS para evitar que tareas recién arrancadas sean sustituidas antes de completar el arranque.

9.2 Logs y métricas

Los logs del contenedor se envían a CloudWatch Logs mediante el driver `awslogs` de ECS. El grupo de logs sigue la convención `/aws/ecs/transitops/dev/api`. Esta integración permite inspeccionar errores de arranque, fallos de configuración, problemas de conexión con RDS y salidas de tareas puntuales como las migraciones.

En Sprint 6 se sustituyó la salida de consola no estructurada por logging JSON mediante `AddJsonConsole`. La configuración incluye scopes, marcas temporales UTC y campos que CloudWatch puede filtrar. También se añadió un middleware de correlación de peticiones:

- acepta `X-Correlation-ID` si el cliente lo proporciona;
- genera un identificador nuevo si el header no existe;
- devuelve siempre `X-Correlation-ID` en la respuesta;

- alinea `HttpContext.TraceIdentifier` con ese valor;
- abre un scope de logging con `CorrelationId`;
- registra por petición método, ruta, código HTTP, duración, usuario y rol cuando existen.

Esto permite seguir una petición concreta desde la respuesta HTTP hasta los logs de CloudWatch. Además, los errores gestionados por el middleware de excepciones heredan el mismo identificador, por lo que el campo de error devuelto al cliente y el log del servidor quedan correlados.

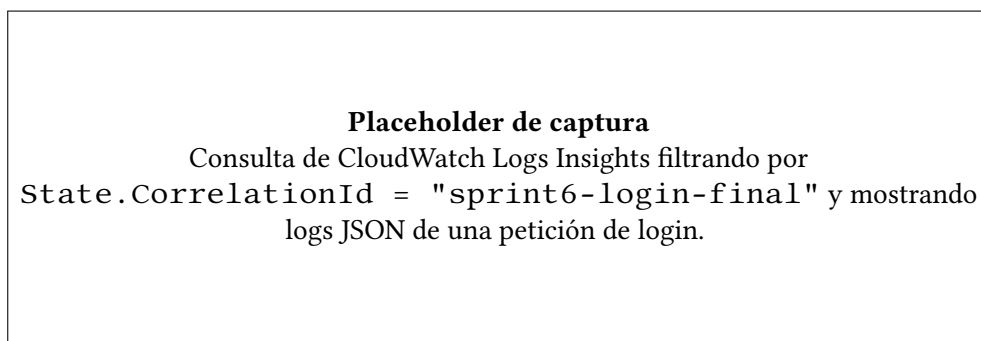


Figura 9.1: Evidencia prevista de logs JSON correlables.

Las métricas básicas de ECS, ALB y RDS quedan disponibles en CloudWatch como parte de los servicios gestionados. El módulo de observabilidad añade un dashboard `transitops-dev-api` con paneles para ALB, utilización de ECS, RDS y logs recientes. El dashboard no sustituye a una plataforma APM completa, pero ofrece una superficie operativa suficiente para un entorno dev defendible.

9.3 Alarmas y operación

La operación de Sprint 6 deja de depender solo de comprobaciones manuales. Terraform crea alarmas CloudWatch para:

- errores de aplicación detectados por filtro métrico sobre logs JSON;
- respuestas 5xx del target group del ALB;
- tiempo de respuesta del ALB;
- ausencia de targets sanos;
- utilización de CPU de ECS;

- utilización de memoria de ECS;
- CPU de RDS;
- conexiones de base de datos;
- almacenamiento libre en RDS.

Las alarmas usan `treat_missing_data` conservador para reducir ruido cuando `dev` está destruido o con `desired_count = 0`. Si se configura `alarm_email`, Terraform crea un topic SNS y una suscripción de email; si no se configura, las alarmas existen sin acciones. Durante la validación de Sprint 6 no se creó topic SNS porque no se proporcionó email.

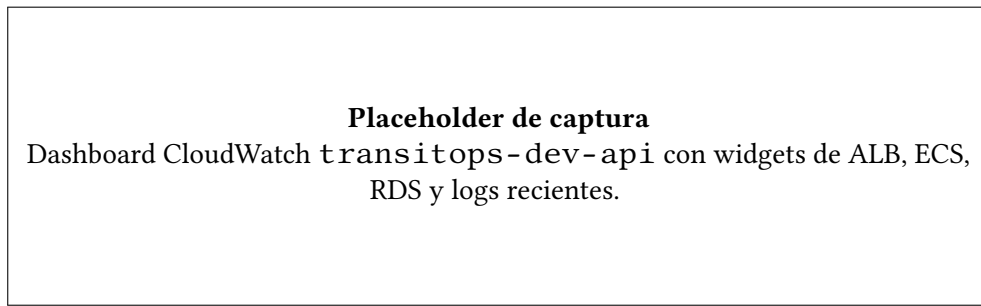


Figura 9.2: Evidencia prevista del dashboard operativo.

9.4 Seguridad aplicada

Las medidas de seguridad aplicadas son:

- Autenticación mediante login y emisión de [JWT](#).
- Contraseñas almacenadas como hash, no en claro.
- Autorización por roles `admin` y `operator`.
- Bootstrap del primer administrador protegido por token externo.
- Secretos en Secrets Manager, no en el repositorio.
- RDS en subredes privadas y sin acceso público.
- ECS en subredes privadas, recibiendo tráfico solo desde ALB.
- ALB como único punto de entrada público.

- HTTPS mediante ACM y redirección HTTP a HTTPS.
- GitHub Actions con OIDC en lugar de claves AWS permanentes.
- Correlación de errores y respuestas mediante `X-Correlation-ID`, evitando exponer trazas internas al cliente.

9.5 Limitaciones de seguridad

El sistema no pretende cubrir todos los controles exigibles a un producto crítico en producción. Quedan fuera del alcance actual la rotación automática de secretos, WAF, protección avanzada contra abuso, auditoría detallada de acciones administrativas, cifrado con claves KMS propias, políticas IAM de mínimo privilegio refinadas al extremo, pruebas de penetración, OpenTelemetry/X-Ray y hardening exhaustivo de cabeceras HTTP.

Estas limitaciones no invalidan el objetivo del trabajo, ya que se ha priorizado una línea base realista y demostrable: autenticación, autorización, red privada, secretos externos, HTTPS e infraestructura reproducible.

Validación y resultados

10.1 Estrategia de validación

La validación se ha organizado en tres niveles: pruebas de backend, validación de infraestructura y pruebas de operación sobre el entorno desplegado. Esta estrategia evita depender de una sola evidencia y permite demostrar tanto comportamiento funcional como capacidad real de despliegue.

En local se ejecutan pruebas automatizadas y flujos Postman/Newman contra una API viva. En infraestructura se usan `terraform fmt`, `terraform validate`, `terraform plan`, `terraform apply` y `terraform destroy`. En AWS se comprueba el estado de ECS, la migración puntual, la salud del endpoint, el bootstrap de administrador, el login y la existencia de recursos de observabilidad.

10.2 Pruebas funcionales

La suite principal de pruebas automatizadas contiene 87 pruebas superadas tras Sprint 6. Estas pruebas cubren endpoints de salud, transportes, vehículos, conductores, autenticación, usuarios, reglas de estado y el contrato de correlación de peticiones. La validación automatizada se complementa con un flujo de smoke local basado en Postman/Newman, que arranca Docker Compose, prepara datos deterministas, ejecuta peticiones y limpia los datos generados.

Además, se verificaron manualmente el bootstrap del primer administrador y el login en el entorno cloud.

10.3 Validación de infraestructura

La infraestructura se validó con Terraform antes y después del despliegue. Las acciones principales fueron:

- Inicialización de backend remoto S3/DynamoDB.
- `terraform validate` correcto para el entorno dev.
- `terraform apply` real de la infraestructura dev.
- Publicación de imagen en ECR.
- Aplicación de migraciones EF Core mediante ECS task.
- Escalado del servicio ECS a una tarea.
- Activación del camino DNS/HTTPS con Route 53 y ACM.
- Creación de dashboard, alarmas y filtro métrico de errores.
- `terraform plan` con `-detailed-exitcode` sin cambios pendientes antes del cierre de coste.
- `terraform destroy` para eliminar recursos costosos del entorno dev.

10.4 Resultados obtenidos

Los resultados técnicos obtenidos en la validación cloud son:

- Endpoint de validación Sprint 6: DNS público del ALB registrado como evidencia operativa.
- Endpoint objetivo DNS/HTTPS: <https://api.dev.transitops.net>, gestionado por Route 53 y ACM cuando el entorno se recrea con `enable_https = true`.
- Health check Sprint 6: `GET /api/v1/health/ready` devuelve 200.
- Certificado ACM: validación DNS corregida en la cuenta 661000947340; el certificado para `api.dev.transitops.net` llegó a estado ISSUED antes del destroy.
- Servicio ECS: `ACTIVE`, `desired = 1`, `running = 1`.
- Imagen ECR desplegada en Sprint 6: `sprint6-local-6bb910c`.

- Task definition desplegada: `transitops-dev-api:3`.
- Tarea ECS de migración: `345ae53d1340475aa94a826328404998`, finalizada con código 0.
- Bootstrap de administrador: respuesta 201.
- Login administrador: respuesta 200 con `JWT`.
- Header de correlación: `X-Correlation-ID` presente en health, errores y login.
- CloudWatch Logs: entradas JSON filtrables por `State.CorrelationId`, incluyendo `sprint6-login-final`.
- Observabilidad: dashboard CloudWatch, log group de la API y 9 alarmas creadas por Terraform.
- SNS: topic no creado porque `alarm_email` no se configuró.
- Cierre de coste: `terraform destroy` eliminó los recursos del entorno dev; el state quedó vacío.

Estos resultados demuestran que el sistema no solo compila, sino que se ejecuta sobre infraestructura cloud real y verificable, y que también puede destruirse de forma controlada para evitar gasto residual.

10.5 Evidencia de cierre de coste

Tras la validación de Sprint 6 se ejecutó `terraform destroy`. El primer intento eliminó la mayoría de recursos y falló al borrar ECR porque el repositorio contenía imágenes. Se vació el repositorio con `aws ecr batch-delete-image` y se repitió el destroy, que finalizó correctamente. Después se verificó que:

- `terraform state list` no devolvía recursos.
- No quedaban ALB, RDS, ECS ni ECR asociados a `transitops-dev`.
- El NAT Gateway aparecía en estado `deleted`.
- No quedaban certificados ACM ni registros Route 53 de `api.dev.transitops.net`.
- No quedaban log groups, dashboard, alarmas ni secretos del entorno dev.

Permanecen fuera de este destroy el dominio registrado `transitops.net`, su hosted zone pública y el backend remoto de Terraform, ya que son recursos de base necesarios para recreaciones posteriores y no forman parte del stack efímero de aplicación.

10.6 Análisis de limitaciones

Las principales limitaciones son:

- No existe interfaz gráfica; el sistema se consume mediante API, Postman o clientes equivalentes.
- El entorno desplegado es `dev`, no una arquitectura productiva multi-entorno completa.
- No se han implementado políticas de autoscaling ni pruebas de carga.
- La observabilidad cubre logs JSON, correlación, dashboard y alarmas básicas, pero no incluye trazas distribuidas ni APM completo.
- El pipeline de GitHub Actions está preparado y ampliado, pero todavía debe archivarse una ejecución completa desde GitHub como evidencia final de CI/CD.
- Algunas políticas IAM son prácticas para el sprint actual y pueden afinarse más en fases posteriores.

Conclusiones

11.1 Conclusiones del trabajo

El trabajo ha alcanzado un estado técnico avanzado respecto a sus objetivos principales. Se ha construido un backend funcional para gestión de transportes y se ha desplegado en AWS mediante infraestructura como código, con base de datos privada, contenedores, secretos externos, configuración DNS/HTTPS, observabilidad en CloudWatch y validación operativa.

El valor principal del proyecto no reside únicamente en la API implementada, sino en haber completado un camino de ingeniería real: código fuente, pruebas, imagen Docker, ECR, Terraform, RDS, ECS, migraciones, DNS, TLS, logs estructurados, correlación de peticiones, dashboard, alarmas, health check, bootstrap, login validado y destrucción controlada del entorno para evitar coste innecesario. Esto permite defender el trabajo como una plataforma cloud operable, no solo como una aplicación local.

11.2 Competencias aplicadas

Durante el desarrollo se han aplicado competencias de:

- Ingeniería de requisitos y control de alcance.
- Diseño de API REST y contratos de error.
- Modelado de dominio y reglas de negocio.
- Persistencia relacional con PostgreSQL y migraciones.
- Pruebas automatizadas e integración.
- Contenerización con Docker.
- Infraestructura como código con Terraform.

- Diseño de red y servicios cloud en AWS.
- CI/CD con GitHub Actions y OIDC.
- Seguridad básica de aplicación e infraestructura.
- Observabilidad con logs estructurados, correlación, métricas y alarmas.
- Operación y control de costes mediante ciclos de creación, validación y destrucción de infraestructura.
- Documentación técnica y preparación de evidencias.

11.3 Lecciones aprendidas

Una lección importante es que la complejidad real de un sistema cloud no aparece solo en el código de negocio. DNS, certificados, roles, secretos, migraciones, estado Terraform, observabilidad, limpieza de recursos y permisos pueden bloquear un despliegue si no se tratan como parte del diseño.

También se ha comprobado el valor de mantener un alcance funcional reducido. Esta decisión permitió dedicar tiempo a cerrar aspectos que a menudo quedan fuera de proyectos académicos: despliegue real, HTTPS, base de datos privada, migraciones controladas, observabilidad, alarmas, rollback de despliegue y validación de extremo a extremo.

Por último, el proyecto muestra que documentar decisiones durante el desarrollo facilita retomar el trabajo, justificar cambios y preparar la defensa.

11.4 Líneas futuras

Las líneas futuras más relevantes son:

- Desarrollar un frontend web para operadores y administradores.
- Ejecutar y archivar el flujo completo de despliegue desde GitHub Actions como evidencia CI/CD final.
- Añadir entorno `prod` separado, con configuración y estado Terraform independientes.
- Refinar permisos IAM hacia mínimo privilegio estricto.
- Evolucionar la observabilidad hacia trazas distribuidas, métricas de negocio y runbooks asociados a cada alarma.
- Incorporar pruebas de carga y análisis de capacidad.

- Implementar rotación de secretos y políticas de backup/restore verificadas.
- Ampliar el dominio con planificación, costes, rutas o integración con servicios externos.

Apéndices

Material complementario

A.1 Anteproyecto

El anteproyecto firmado se conserva en el repositorio de la memoria como referencia documental del alcance aprobado.

A.2 Evidencias técnicas

Las siguientes evidencias deberían incorporarse como capturas o anexos finales antes de la entrega:

Cuadro A.1: Capturas recomendadas para la memoria

Archivo sugerido	Contenido recomendado
imaxes/ aws-ecs-service-sprint6. png	Servicio ECS transitops-dev-api-svc mostrando estado ACTIVE, desired 1 y running 1 durante la validación de Sprint 6.
imaxes/ aws-target-group-healthy. png	Target group del ALB mostrando la tarea ECS como healthy usando /api/v1/health/ready.
imaxes/ aws-alb-listeners-https. png	Listeners del ALB con puerto 80 redirigiendo a HTTPS y puerto 443 usando el certificado ACM, en la recreación con enable_https = true.

<code>imaxes/aws-acm-issued.png</code>	Certificado ACM de <code>api.dev.transitops.net</code> en estado Issued.
<code>imaxes/aws-route53-records.png</code>	Hosted zone <code>transitops.net</code> con registro <code>api.dev.transitops.net</code> y CNAME de validación ACM.
<code>imaxes/aws-cloudwatch-dashboard.png</code>	Dashboard CloudWatch <code>transitops-dev-api</code> con widgets de ALB, ECS, RDS y logs recientes.
<code>imaxes/aws-cloudwatch-alarms.png</code>	Lista de las 9 alarmas <code>transitops-dev-api-*</code> creadas por Terraform.
<code>imaxes/aws-cloudwatch-correlation-logs.png</code>	Consulta de CloudWatch Logs Insights filtrando por un <code>CorrelationId</code> , por ejemplo <code>sprint6-login-final</code> .
<code>imaxes/aws-ecr-image-sprint6.png</code>	Repositorio ECR <code>transitops/api</code> mostrando la imagen <code>sprint6-local-6bb910c</code> .
<code>imaxes/health-ready-sprint6.png</code>	Salida de terminal o navegador mostrando <code>/api/v1/health/ready</code> con respuesta 200 y header <code>X-Correlation-ID</code> .
<code>imaxes/login-admin-sprint6.png</code>	Respuesta 200 de login administrador en cloud con JWT y correlación de petición.
<code>imaxes/github-actions-deploy.png</code>	Ejecución verde del workflow <code>deploy-dev.yml</code> , cuando se ejecute desde GitHub Actions.
<code>imaxes/terraform-no-changes.png</code>	Salida de <code>terraform plan</code> indicando que la infraestructura no tiene cambios pendientes antes del cierre de coste.

<code>imagenes/</code>	Evidencia posterior a <code>terraform</code>
<code>terraform-destroy-empty-state.png</code>	<code>terraform destroy</code> : state vacío y ausencia de ALB, RDS, ECS, ECR, ACM, registros DNS del subdominio, logs, dashboard, alarmas y secretos del entorno dev.

Estas capturas no se incluyen todavía para evitar referenciar ficheros inexistentes en LaTeX. La memoria contiene placeholders descriptivos en las secciones donde esas evidencias aportan más valor; cuando se incorporen las imágenes reales, deberán sustituirse los placeholders por `\includegraphics` o mantenerse como anexo de evidencias.

Lista de acrónimos

API Application Programming Interface. [2](#)

AWS Amazon Web Services. [1](#)

CI/CD Continuous Integration / Continuous Deployment. [2](#)

ECR Elastic Container Registry. [14](#)

ECS Elastic Container Service. [14](#)

JWT JSON Web Token. [10](#), [12](#), [18](#), [30](#), [34](#)

RDS Relational Database Service. [14](#)

Glosario

backend Componente servidor responsable de exponer la lógica de negocio, gestionar la persistencia y ofrecer una interfaz de comunicación a otros clientes o sistemas.. [2](#)

infraestructura como código Práctica consistente en definir recursos de infraestructura mediante ficheros versionables y automatizables.. [1](#), [3](#)

Bibliografía

- [1] HashiCorp, “Terraform documentation,” 2026, consultado el 2 de mayo de 2026. [En línea]. Disponible en: <https://developer.hashicorp.com/terraform/docs>
- [2] Microsoft, “Asp.net core documentation,” 2026, consultado el 2 de mayo de 2026. [En línea]. Disponible en: <https://learn.microsoft.com/aspnet/core/>
- [3] Amazon Web Services, “Amazon elastic container service documentation,” 2026, consultado el 2 de mayo de 2026. [En línea]. Disponible en: <https://docs.aws.amazon.com/ecs/>
- [4] —, “Amazon rds documentation,” 2026, consultado el 2 de mayo de 2026. [En línea]. Disponible en: <https://docs.aws.amazon.com/rds/>
- [5] —, “Amazon route 53 documentation,” 2026, consultado el 2 de mayo de 2026. [En línea]. Disponible en: <https://docs.aws.amazon.com/route53/>
- [6] —, “Aws certificate manager documentation,” 2026, consultado el 2 de mayo de 2026. [En línea]. Disponible en: <https://docs.aws.amazon.com/acm/>